

UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

Universidad Técnica Federico Santa María

Departamento de Industrias

ICN-292 — Sistemas de Información para la Gestión

Laboratorio 5: SQL y Bases de Datos Creación de la Base y Consultas

Andes Bike

Creación del modelo relacional, carga oficial y diez consultas de validación

Autor:	Matías Gómez
Rol USM:	202360624-3
Paralelo:	100
Profesores:	José Miguel González Paul
	José Luis Sáez Tamayo
	Patricio Cifuentes
Repositorio GitHub:	`https://github.com/Maty0401/ICN292-Lab5-Gomez-Matias`
Fecha:	11 de noviembre de 2026

Consultas SQL

A continuación se presenta, para cada una de las diez consultas solicitadas, el código SQL tal como fue ejecutado y la captura del resultado devuelto por MySQL Workbench sobre la carga oficial de la base `andesbike`.

Consulta 1

Liste todos los vendedores con la cantidad de ventas que registraron, incluidos los que no registraron ninguna. Ordene de mayor a menor cantidad y, a igualdad, por nombre. Los que no tienen ventas deben mostrar cero, no quedar en blanco.

```sql
SELECT   v.nombre                  AS vendedor,
         COUNT(ve.id_venta)        AS cantidad_ventas
FROM        vendedores v
LEFT JOIN   ventas     ve   ON ve.id_vendedor = v.id_vendedor
GROUP BY    v.id_vendedor, v.nombre
ORDER BY    cantidad_ventas DESC, v.nombre ASC;
```

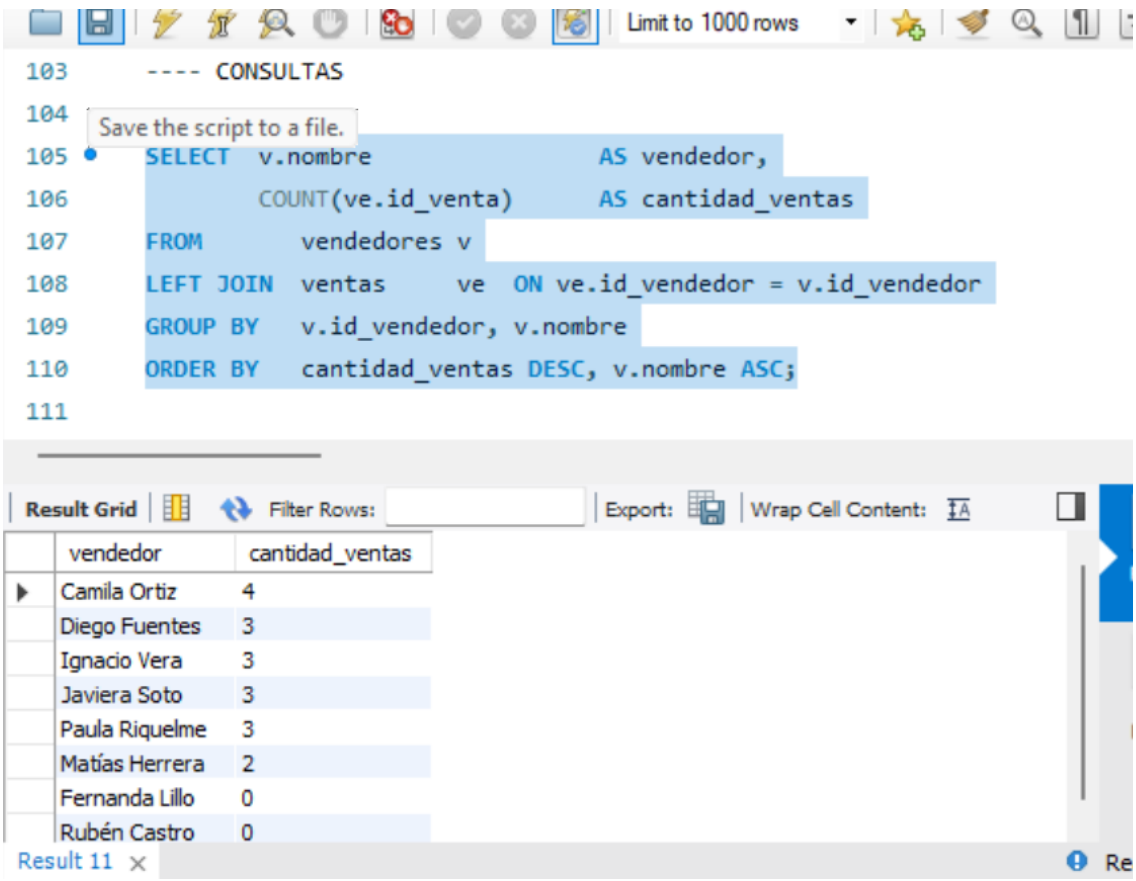


Figura 1: Resultado de la Consulta 1 en MySQL Workbench.

Consulta 2

Para cada tienda muestre su nombre, cuántas ventas se registraron y cuántas unidades se vendieron, incluidas las tiendas que no tienen ningún vendedor. Ordene por unidades de mayor a menor.

```sql
SELECT  t.nombre                              AS tienda,
        COUNT(ve.id_venta)                    AS ventas_registradas,
        COALESCE(SUM(ve.unidades), 0)         AS unidades_vendidas
FROM       tiendas    t
LEFT JOIN  vendedores v   ON v.id_tienda    = t.id_tienda
LEFT JOIN  ventas     ve  ON ve.id_vendedor = v.id_vendedor
GROUP BY   t.id_tienda, t.nombre
ORDER BY   unidades_vendidas DESC;
```

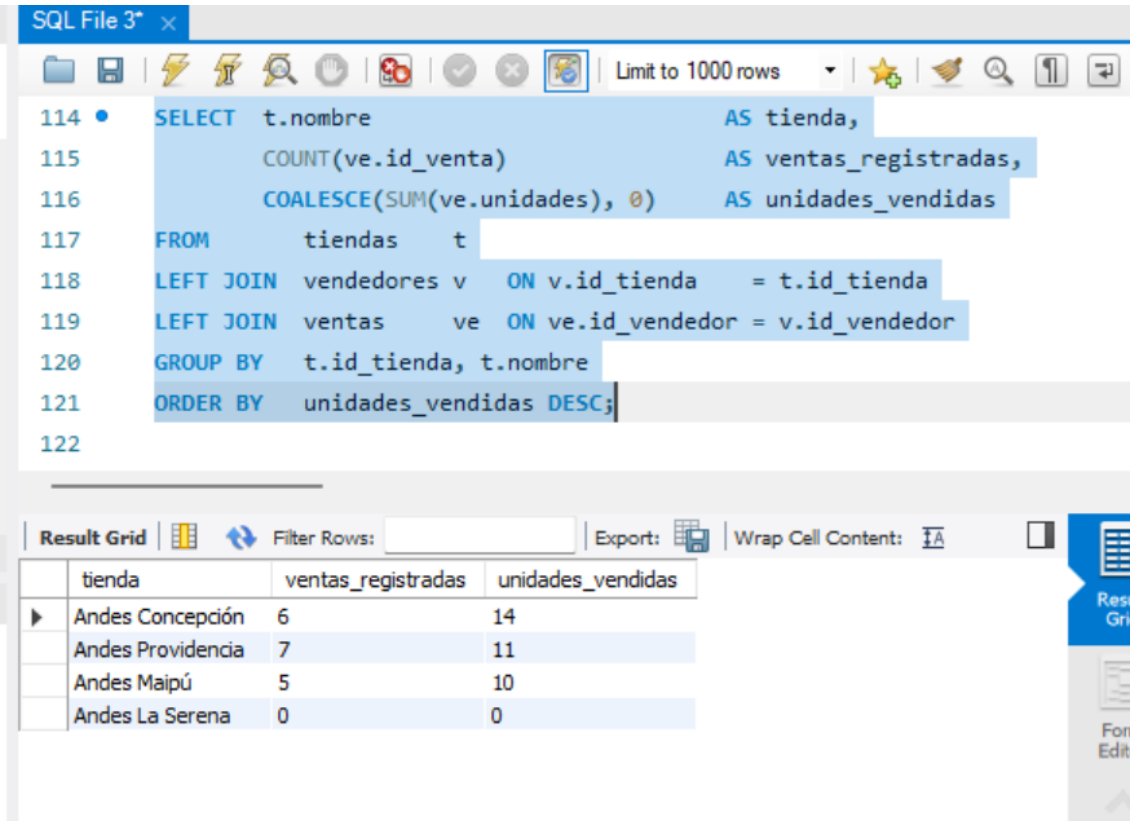


Figura 2: Resultado de la Consulta 2 en MySQL Workbench.

Consulta 3

Liste las tiendas que no tienen ningún vendedor asignado.

```sql
SELECT  t.id_tienda,
        t.nombre   AS tienda,
        t.comuna
FROM       tiendas    t
LEFT JOIN  vendedores v  ON v.id_tienda = t.id_tienda
```

```
WHERE       v.id_vendedor IS NULL
ORDER BY    t.nombre;
```

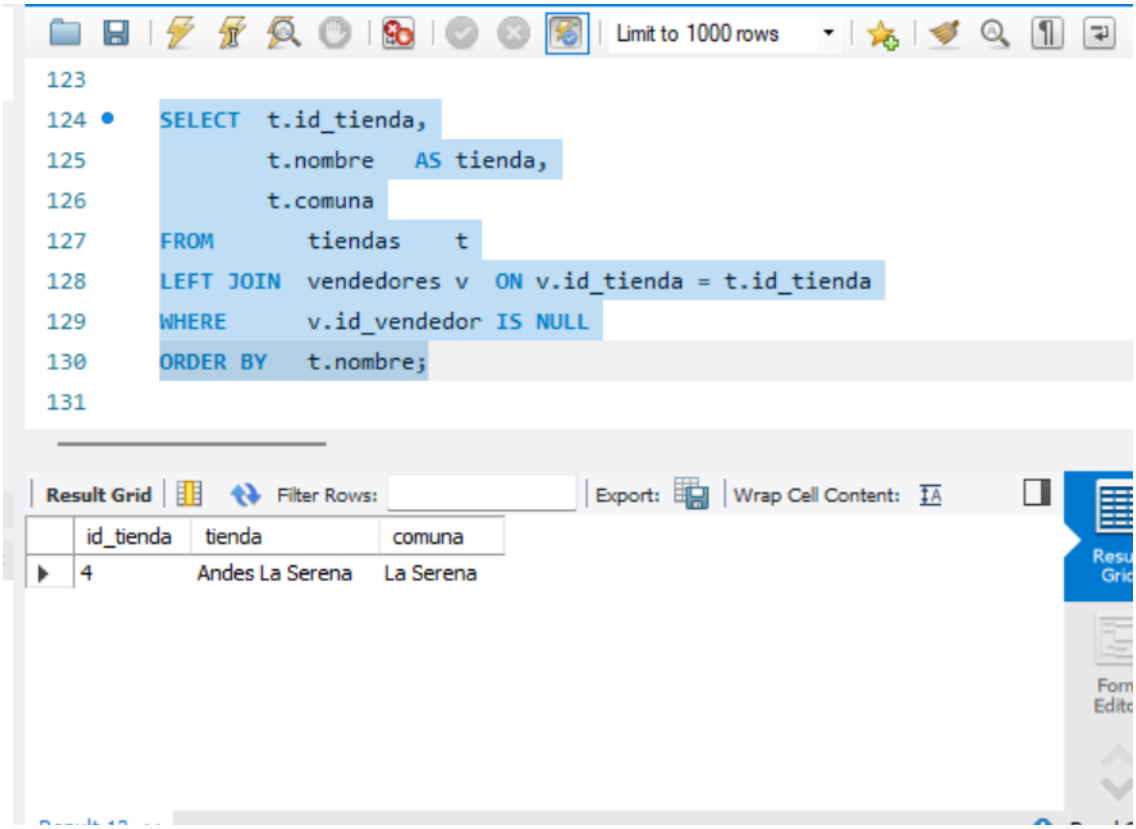


Figura 3: Resultado de la Consulta 3 en MySQL Workbench.

Consulta 4

Liste los modelos que no se vendieron ni una sola vez en junio de 2025.

```
SELECT   b.modelo,
         b.categoria,
         b.precio
FROM        bicicletas b
LEFT JOIN   ventas     ve
       ON   ve.id_bicicleta = b.id_bicicleta
      AND   ve.fecha >= '2025-06-01'
      AND   ve.fecha  < '2025-07-01'
WHERE       ve.id_venta IS NULL
ORDER BY    b.modelo;
```

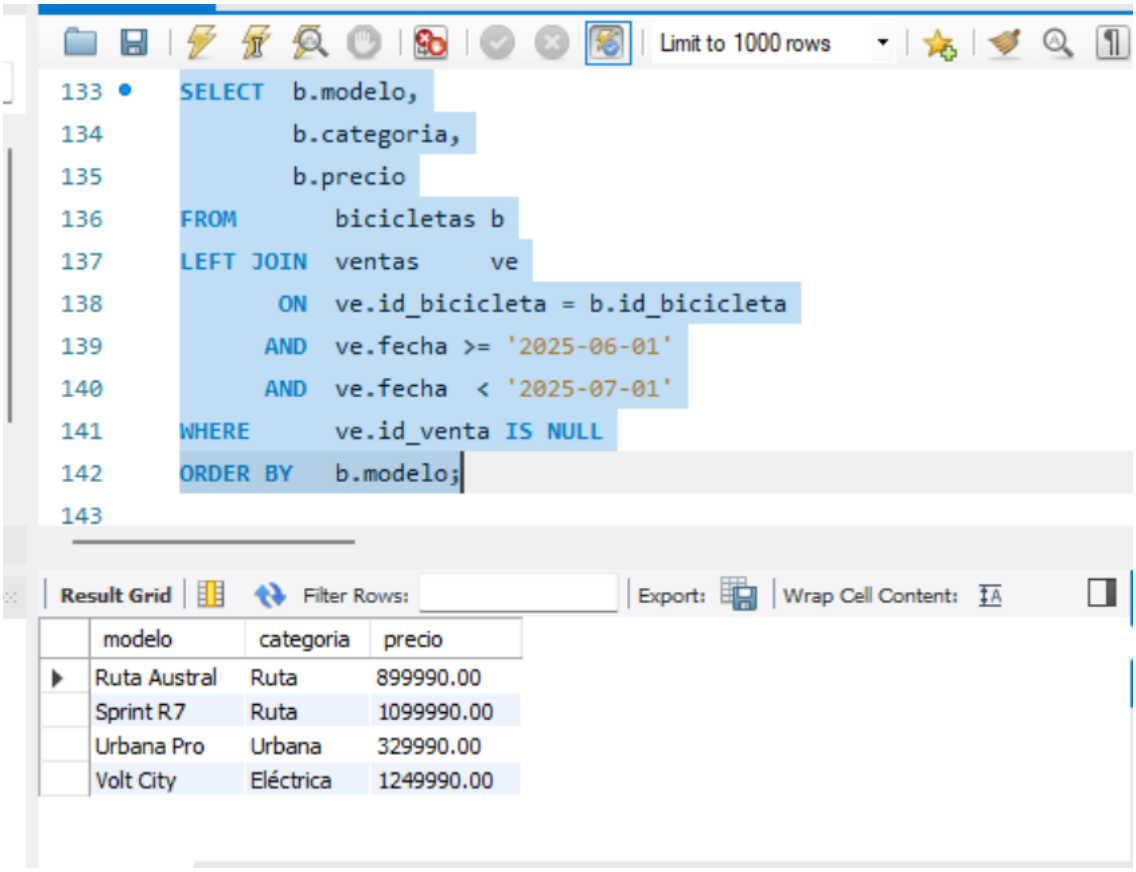


Figura 4: Resultado de la Consulta 4 en MySQL Workbench.

Ubicación de la condición del mes: la condición de junio de 2025 está puesta en el `ON` del `LEFT JOIN`, de modo que el join trae únicamente las ventas de ese mes y el `WHERE ve.id_venta IS NULL` deja los modelos que no tuvieron ninguna; si esa misma condición se hubiera puesto en el `WHERE`, se evaluaría después del join sobre filas ya rellenadas con `NULL`, chocaría con la condición `IS NULL` y la consulta habría devuelto cero filas.

Consulta 5

Liste los vendedores que no han registrado ninguna venta, mostrando también el nombre de su tienda. Ninguno de ellos puede quedar fuera del resultado.

```sql
SELECT  v.nombre                                          AS vendedor,
        COALESCE(t.nombre, '(sin tienda asignada)')       AS tienda,
        v.fecha_ingreso
FROM       vendedores v
LEFT JOIN  tiendas    t   ON t.id_tienda    = v.id_tienda
LEFT JOIN  ventas     ve  ON ve.id_vendedor = v.id_vendedor
WHERE      ve.id_venta IS NULL
ORDER BY   v.nombre;
```

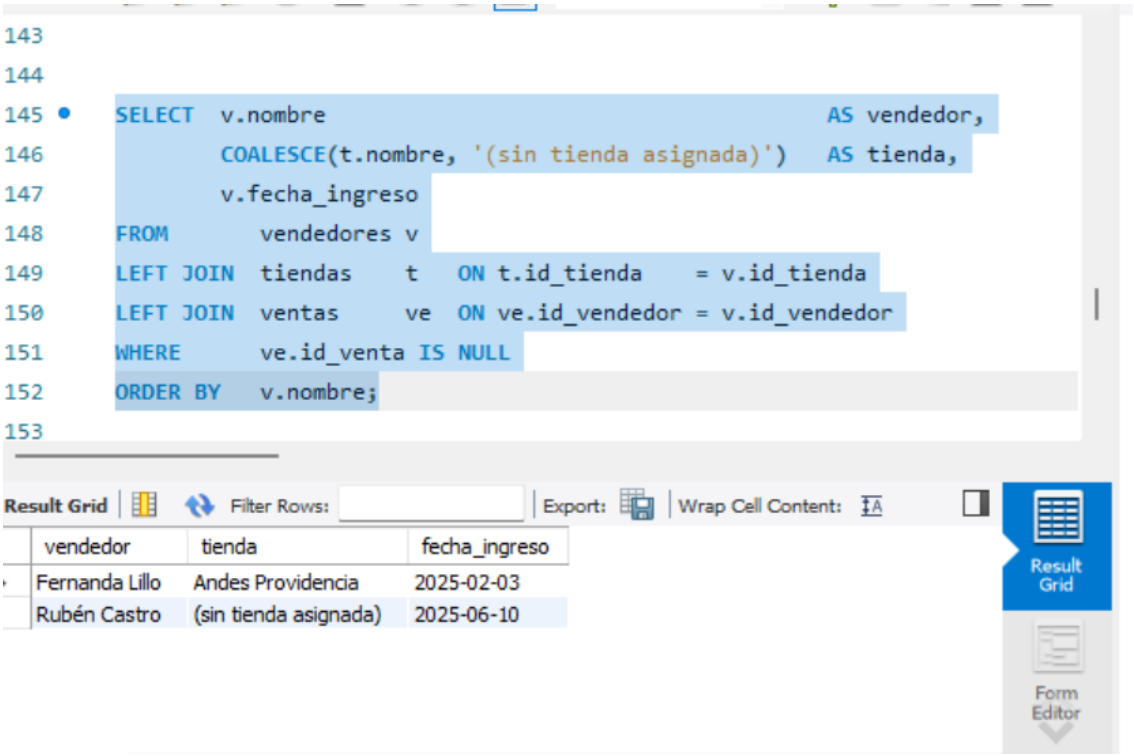


Figura 5: Resultado de la Consulta 5 en MySQL Workbench.

Consulta 6

Liste los pares de vendedores que trabajan en la misma tienda, poniendo primero al que ingresó antes. Muestre los dos nombres, las dos fechas de ingreso y la tienda. Cada par debe aparecer una sola vez.

```sql
SELECT  t.nombre          AS tienda,
        a.nombre          AS vendedor_mas_antiguo,
        a.fecha_ingreso   AS ingreso_mas_antiguo,
        b.nombre          AS vendedor_mas_nuevo,
        b.fecha_ingreso   AS ingreso_mas_nuevo
FROM  vendedores a
JOIN  vendedores b
   ON  a.id_tienda = b.id_tienda
  AND (a.fecha_ingreso <  b.fecha_ingreso
   OR (a.fecha_ingreso =  b.fecha_ingreso
       AND a.id_vendedor < b.id_vendedor))
JOIN  tiendas t ON t.id_tienda = a.id_tienda
ORDER BY t.nombre, a.fecha_ingreso, b.fecha_ingreso;
```

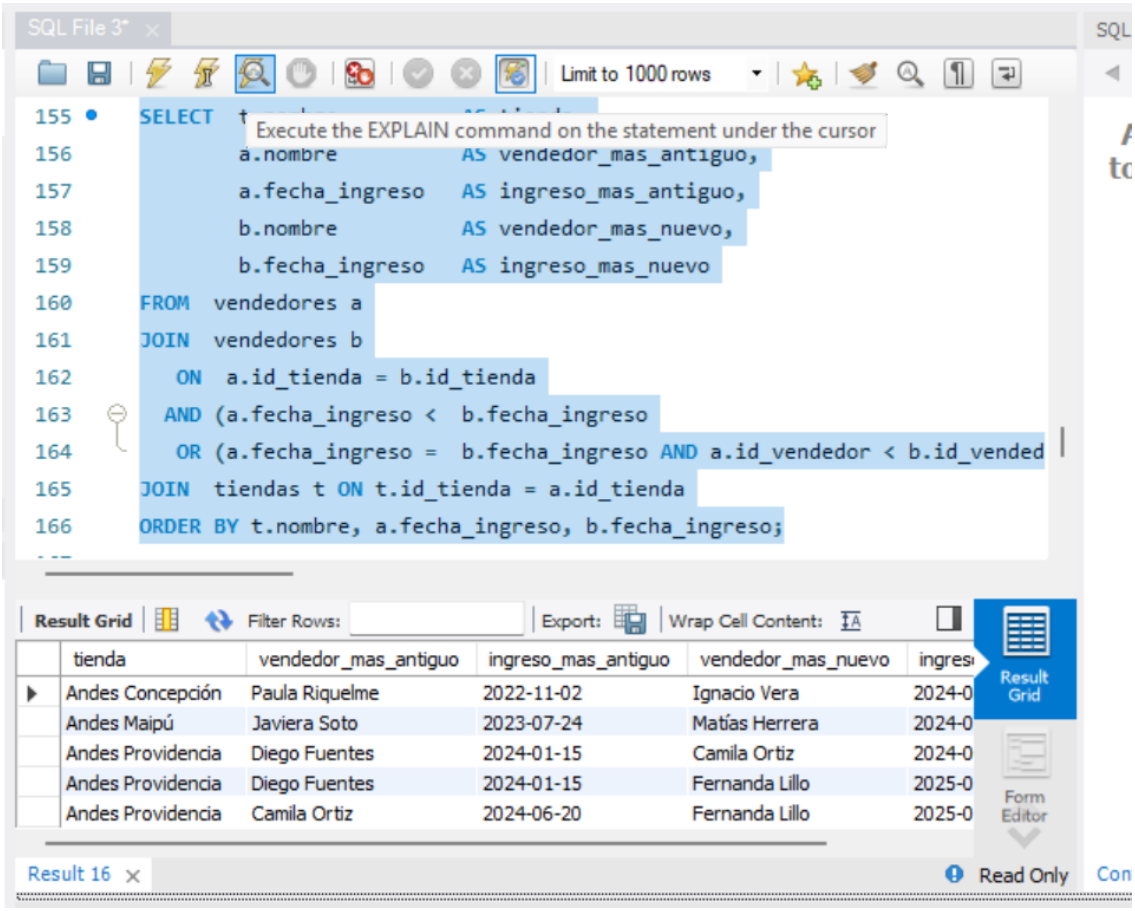


Figura 6: Resultado de la Consulta 6 en MySQL Workbench.

Consulta 7

Para cada tienda, calcule el monto total vendido en bicicletas de las categorías `'Montaña'` y `'Ruta'`, dejando solo las tiendas que superan los $2,000,000. Ordene de mayor a menor.

```
SELECT  t.nombre                          AS tienda,
        SUM(ve.unidades * b.precio)       AS monto_total
FROM   tiendas    t
JOIN   vendedores v   ON v.id_tienda     = t.id_tienda
JOIN   ventas     ve  ON ve.id_vendedor  = v.id_vendedor
JOIN   bicicletas b   ON b.id_bicicleta  = ve.id_bicicleta
WHERE      b.categoria IN ('Montaña', 'Ruta')
GROUP BY   t.id_tienda, t.nombre
HAVING     monto_total > 2000000
ORDER BY   monto_total DESC;
```

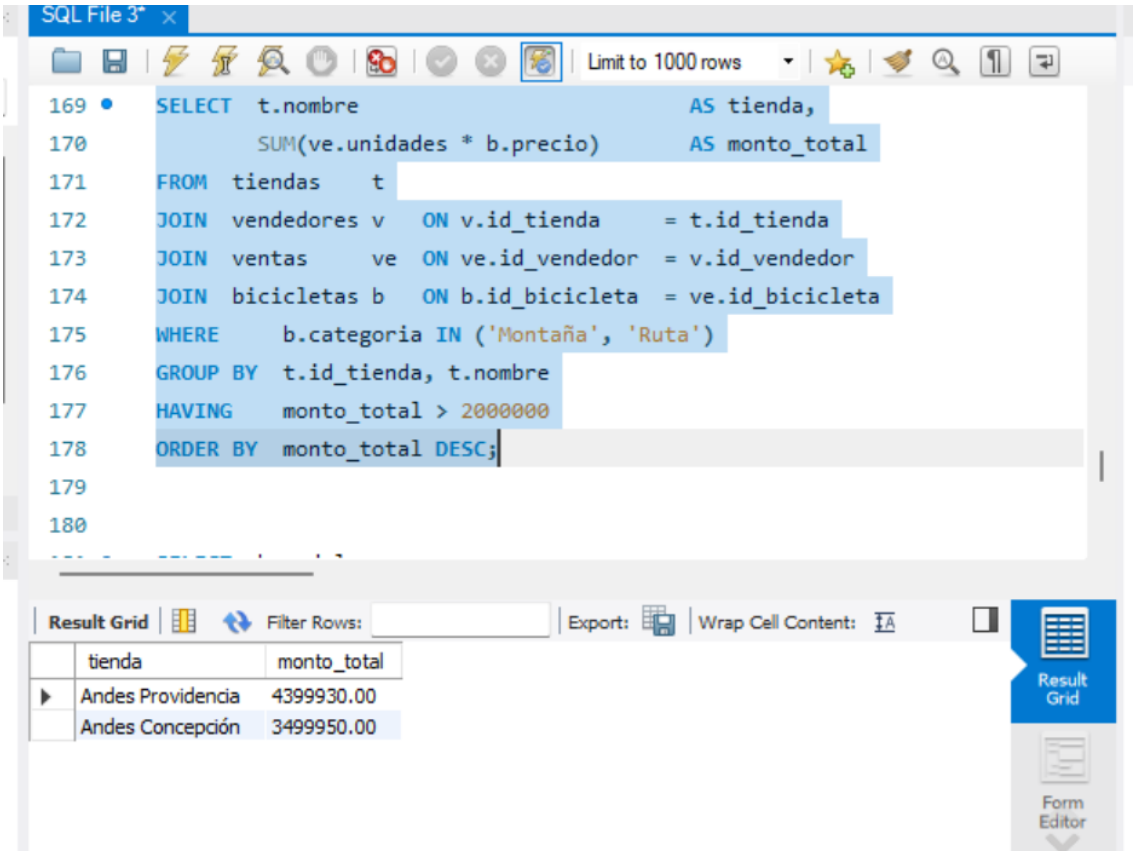


Figura 7: Resultado de la Consulta 7 en MySQL Workbench.

Consulta 8

Liste el modelo y el precio de las bicicletas cuyo precio supera el promedio del catálogo, de mayor a menor.

```
SELECT  b.modelo,
        b.precio
FROM    bicicletas b
WHERE   b.precio > (SELECT AVG(precio) FROM bicicletas)
ORDER BY b.precio DESC;
```

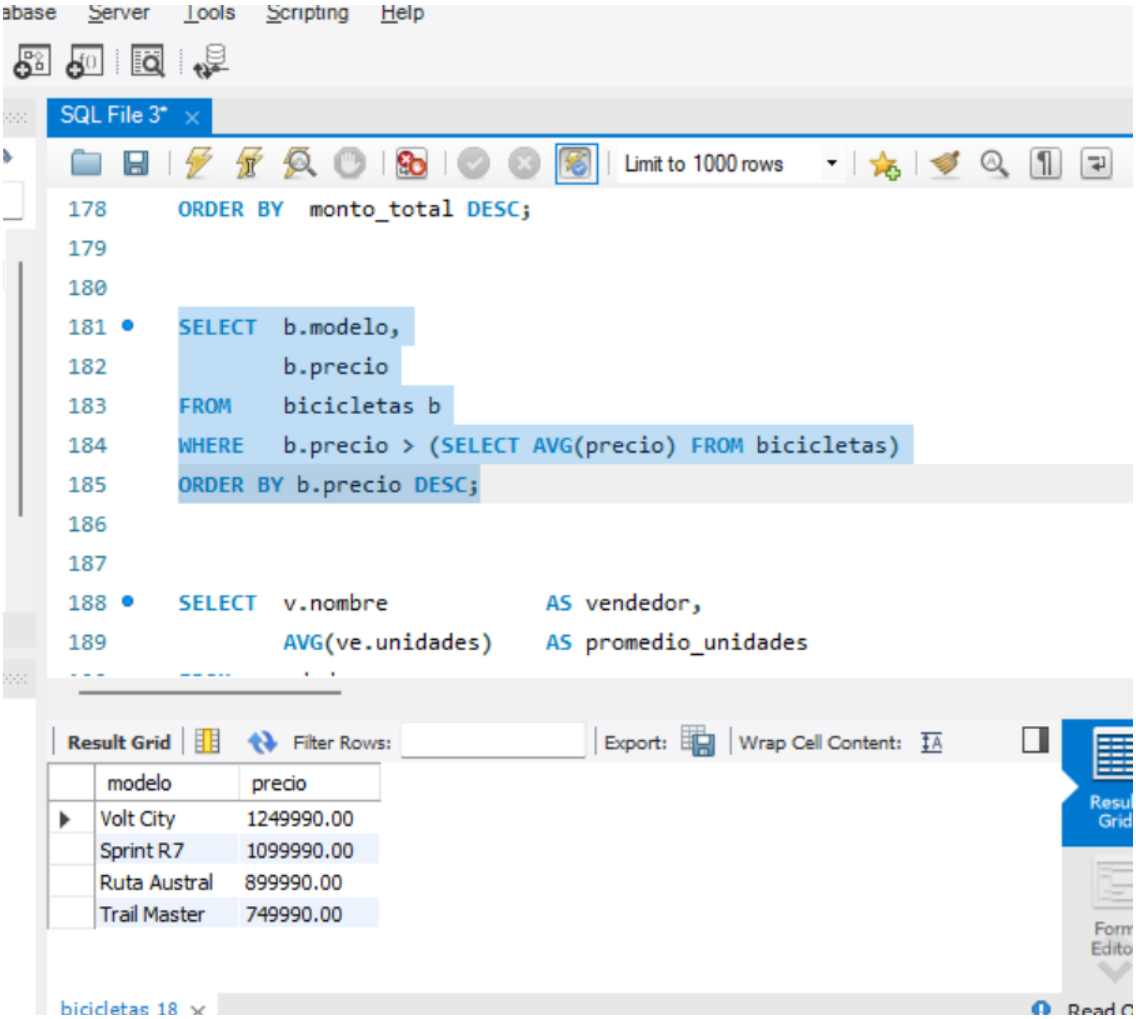


Figura 8: Resultado de la Consulta 8 en MySQL Workbench.

Valor del promedio y su origen: el promedio del catálogo es \$657,490,00 y se obtuvo con la subconsulta escalar `SELECT AVG(precio) FROM bicicletas`, es decir, promediando los ocho modelos del catálogo (5,259,920 / 8 = 657,490) y no las unidades efectivamente vendidas.

Consulta 9

Liste los vendedores cuyo promedio de unidades por venta supera el promedio de unidades de todas las ventas de la cadena. Muestre el nombre y su promedio, de mayor a menor y, a igualdad, por nombre.

```
SELECT  v.nombre            AS vendedor,
        AVG(ve.unidades)    AS promedio_unidades
FROM  vendedores v
JOIN  ventas     ve  ON ve.id_vendedor = v.id_vendedor
GROUP BY  v.id_vendedor, v.nombre
HAVING    AVG(ve.unidades) > (SELECT AVG(unidades) FROM ventas)
ORDER BY  promedio_unidades DESC, v.nombre ASC;
```

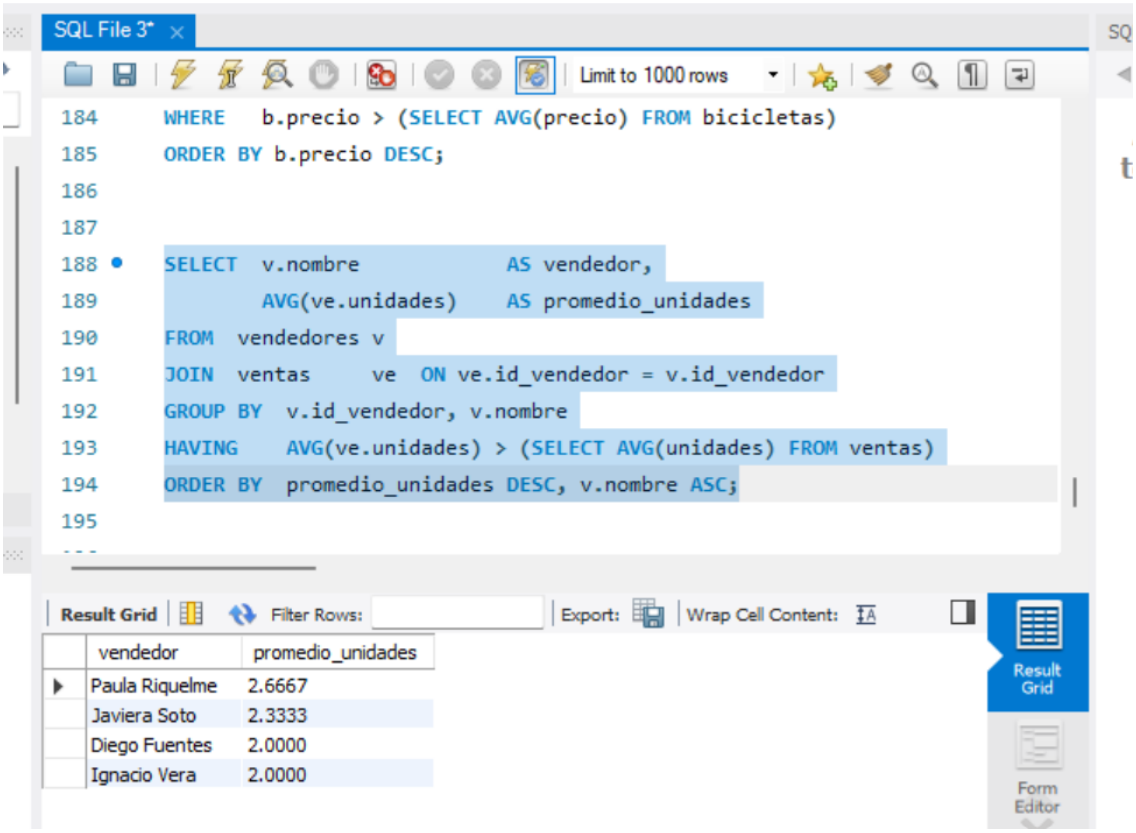


Figura 9: Resultado de la Consulta 9 en MySQL Workbench.

Consulta 10

Liste todos los vendedores con el monto total que vendieron, incluidos los que no vendieron nada, de mayor a menor monto.

```
SELECT  v.nombre                          AS vendedor,
        SUM(ve.unidades * b.precio)       AS monto_total
FROM        vendedores v
LEFT JOIN   ventas     ve  ON ve.id_vendedor  = v.id_vendedor
LEFT JOIN   bicicletas b   ON b.id_bicicleta  = ve.id_bicicleta
GROUP BY    v.id_vendedor, v.nombre
ORDER BY    monto_total DESC;
```

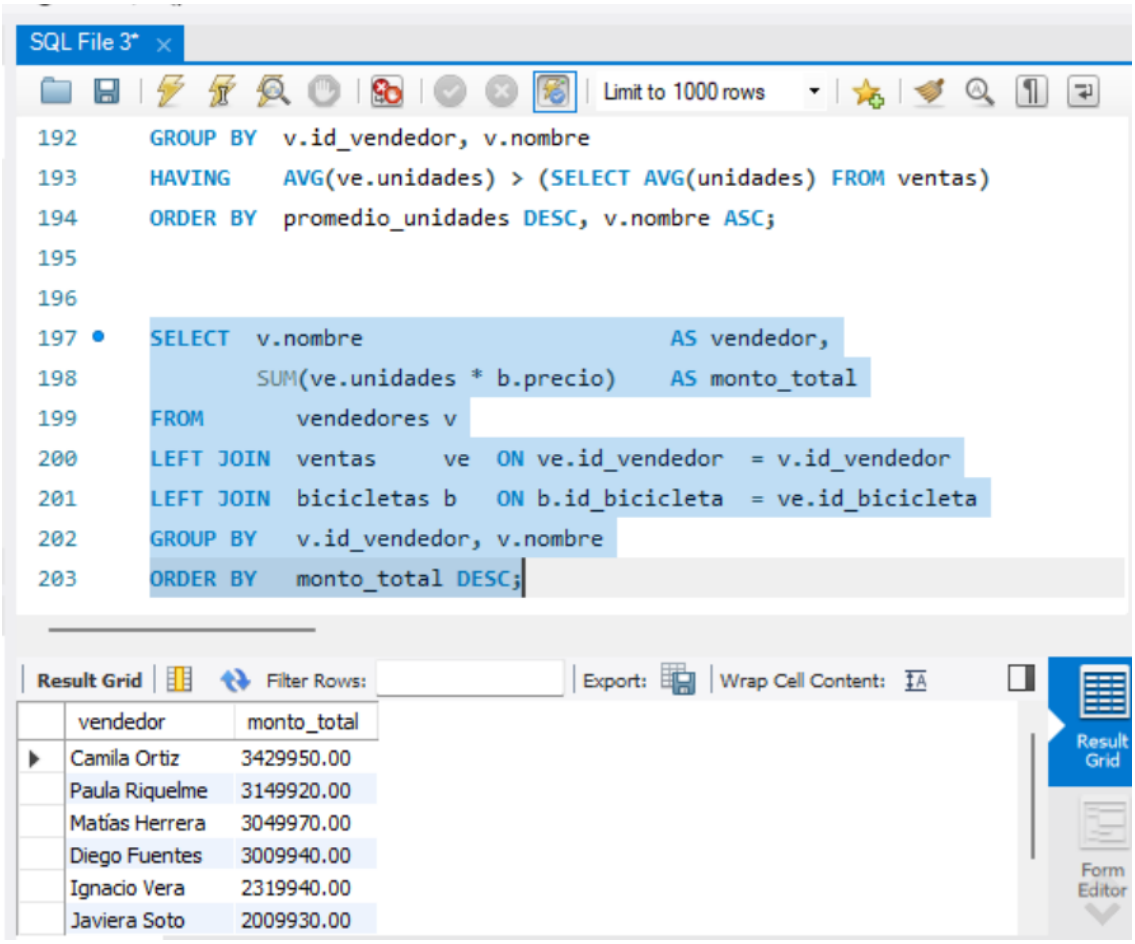


Figura 10: Resultado de la Consulta 10 en MySQL Workbench.

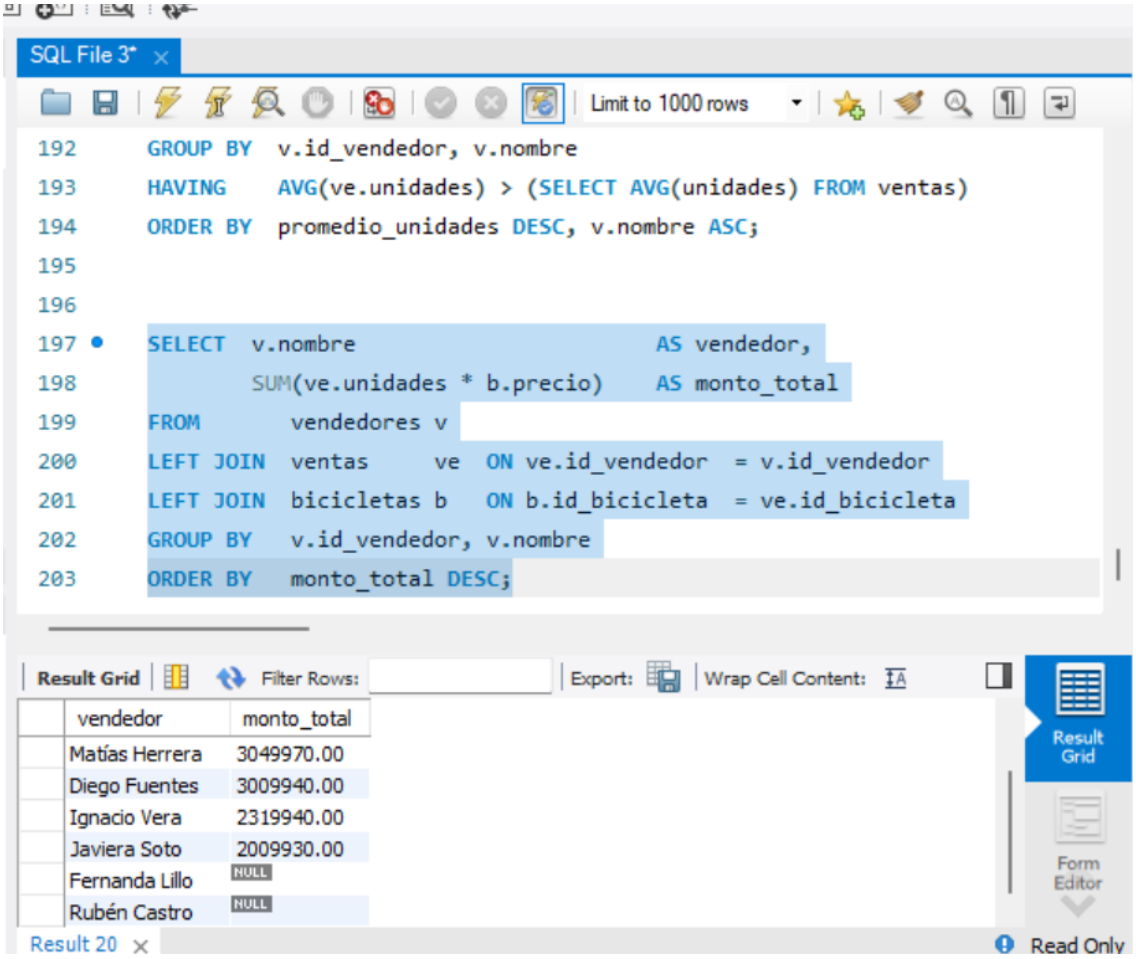


Figura 11: Resultado de la Consulta 10 en MySQL Workbench (continuación).

Por qué aparecen `NULL` y no cero: Fernanda Lillo y Rubén Castro no muestran cero sino `NULL` porque el `LEFT JOIN` sí los conserva, pero no tienen ninguna fila de venta que sumar y `SUM()` sobre un conjunto vacío devuelve `NULL`; para que mostraran cero habría que agregar `COALESCE` a la suma, escribiendo `COALESCE(SUM(ve.unidades * b.precio), 0) AS monto_total`.